Tuya-Messwerte

Lernunterlage: Watt, Volt und Akku – von der rohen Tuya-Zahl bis zur Anzeige in der Kachel

1 Was neu ist

- **Geräte-Kachel:** neue Zeilen „Messwerte“ (z. B. „0,0 W · 227 V“ oder „Akku 83 %“) und „Gemessen“ mit Uhrzeit. Ist das Gerät offline, steht dahinter „(Gerät offline, letzter Stand)“.
- **Raumkarte auf der Übersicht:** Zustand und Messwerte in einer Zeile, z. B. „ZU · Akku 83 %“.
- **Warnung:** Ab 20 % Akku wird der Wert gelb. So tauscht man die Batterie, bevor der Türsensor ausfällt.
- **Ausgelöst** wird alles durch „Mit Tuya abgleichen“ auf der Geräteseite (`POST /api/tuya/sync`).

2 Rohwerte und Skalierung

Tuya schickt Messwerte als **ganze Zahlen**. Wie viele Nachkommastellen gemeint sind, sagt ein **Skalierungsfaktor**. Das spart Datenmenge und vermeidet Rundungsfehler, die bei Kommazahlen in JSON entstehen können. Umrechnen muss der Empfänger, also wir.

Die Tabelle zeigt die echten Rohwerte unserer Geräte vom 02.11.2026 (`./tuya-rohwerte-anzeigen.sh`):

Code	Rohwert	Rechnung	Anzeige	Wie sicher?
cur_voltage	2271	2271 ÷ 10	227 V	geprüft: passt zur Netzspannung
cur_power	0	Wert ÷ 10	0,0 W	Tuya-Konvention, mit bekannter Last prüfen
battery_percentage	83	unverändert	Akku 83 %	geprüft
cur_current	0	–	nicht angezeigt	Milliampere, für uns ohne Mehrwert
va_temperature	–	–	–	kein Gerät liefert ihn bisher

> Den Faktor nie raten, sondern an echten Daten prüfen. Ein falscher Faktor zeigt 2271 V oder 22,7 V an, und keiner merkt es, solange niemand hinschaut.

3 Der Weg eines Messwerts

```
Steckdose ──WLAN──► Tuya-Cloud          (merkt sich den letzten Stand)
                       │
POST /api/tuya/sync    ▼
TuyaDeviceSyncer  ── GET /v1.0/devices/{id}/status ──►  [{code:"cur_voltage", value:2271}, …]
       │
       ├─ Online? (Geräteliste aus dem 60-s-Cache, derselbe wie für die LEDs)
       ├─ ApplyMeasurements: 2271 ÷ 10 → VoltageVolt = 227.1, MeasuredAtUtc = jetzt
       └─ SaveChanges → Tabelle Devices
                       │
GET /api/devices       ▼
Frontend  measurementsOf(device) → "227 V"  → Kachel und Raumkarte
```

4 Der Code im Backend

Zahlen sicher lesen

```
public static long? GetNumber(List<TuyaStatusItem> status, string code)
{
    var item = status.FirstOrDefault(s => s.Code == code);
    if (item is null || item.Value.ValueKind != JsonValueKind.Number)
        return null;                        // fehlt, Text "83" oder true → nichts raten
    return item.Value.TryGetInt64(out var number) ? number : null;   // 12.5 → null
}
```

Messwerte übernehmen

```
if (GetNumber(status, VoltageCode) is { } voltage && voltage >= 0)
{
    device.VoltageVolt = voltage / VoltageScale;     // 2271 / 10.0 = 227.1
    found = true;
}
if (GetNumber(status, BatteryCode) is { } battery && battery is >= 0 and <= 100)
{
    device.BatteryPercent = (int)battery;
    found = true;
}
if (found) device.MeasuredAtUtc = timestamp;
```

- `is { } voltage` ist ein Muster (Pattern Matching): „ist nicht null, und nenne den Wert voltage“. Spart eine eigene Zeile mit `.Value`.
- `battery is >= 0 and <= 100` prüft den Bereich in einem Ausdruck. Ein Akku mit 150 % ist ein Fehler, kein Messwert.
- **Fester Faktor als Konstante** (`PowerScale = 10`), nicht als Zahl mitten im Code. So steht an einer Stelle mit Kommentar, woher er kommt.

5 Zustand oder Ereignis?

	Ereignis (DeviceEvents)	**Letzter Stand (Spalten am Gerät)**
Beispiel	Licht an, Tür auf	123,4 W, Akku 83 %
Wann gespeichert	nur bei einem Wechsel	bei jedem Abgleich überschrieben
Verlauf	ja, mit Zeitpunkt	nein, nur der neueste Wert
Warum so	Wechsel sind die Aussage („wann war die Tür offen?“)	Watt schwankt ständig, der Verlauf wäre voller Einträge ohne Aussage

Ein Verlauf für Diagramme („Verbrauch pro Tag“) ist bewusst **nicht** gebaut. Dafür bräuchte es eine eigene Tabelle und einen Hintergrunddienst, der regelmäßig abfragt. Das ist ein eigener Schritt.

6 Offline-Geräte: der letzte Stand ist nicht neu

Ist eine Steckdose offline, antwortet die Tuya-Cloud trotzdem, nur eben mit dem **zuletzt bekannten** Stand. Würden wir den übernehmen, stünde „gemessen um 11:53“ an einem Wert von gestern.

```
var online = apply ? await LoadOnlineAsync(ct) : null;      // eine Abfrage für alle Geräte
...
var measured = online != false && ApplyMeasurements(device, status, timestamp);
```

- `online != false` bei `bool?`: true und null (unbekannt) gehen durch, nur ein sicheres „offline“ sperrt. Ist die Geräteliste nicht abrufbar, wird gespeichert wie bisher.
- **Kein Zusatz-Kontingent:** Der Online-Zustand kommt aus dem `TuyaOnlineCache` der Status-LEDs (60 Sekunden). Zwei Funktionen teilen sich einen Aufruf.
- **Die Oberfläche** zeigt bei roter LED „(Gerät offline, letzter Stand)“. Ehrlich statt hübsch.

7 Migration: neue Spalten

Neue Eigenschaften am Modell `Device` brauchen neue Spalten. Erzeugt mit `dotnet ef migrations add TuyaMesswerteErgaenzt`, angewendet beim Start:

```
ALTER TABLE "Devices" ADD "BatteryPercent" INTEGER NULL;
ALTER TABLE "Devices" ADD "MeasuredAtUtc"  TEXT    NULL;
ALTER TABLE "Devices" ADD "PowerWatt"      REAL    NULL;
ALTER TABLE "Devices" ADD "VoltageVolt"    REAL    NULL;
```

- **Alle nullable** (`double?`, `int?`): Bestehende Geräte bekommen einfach null. Kein Datenverlust, keine Standardwerte, die wie echte Messungen aussehen.
- **Abwärtskompatibel:** Der alte Dienst lief weiter, obwohl die Datenbank schon die neuen Spalten hatte. EF Core liest nur die Spalten, die es kennt.
- **SQLite** kennt kein DateTimeOffset, EF Core speichert es als Text (TEXT).
- **Neu verknüpft** = Messwerte zurück auf null (`DeviceEndpoints`), damit kein Wert vom falschen Gerät stehen bleibt.

8 Frontend: Zahlen auf Deutsch

```
const wattFormat = new Intl.NumberFormat('de-DE',
  { minimumFractionDigits: 1, maximumFractionDigits: 1 })

export function measurementsOf(device: Device): Measurement[] {
  const result: Measurement[] = []
  if (device.powerWatt !== null)
    result.push({ text: `${wattFormat.format(device.powerWatt)} W`, warning: false })
  ...
  if (device.batteryPercent !== null)
    result.push({ text: `Akku ${device.batteryPercent} %`,
                  warning: device.batteryPercent <= lowBatteryPercent })
  return result
}
```

- **Intl.NumberFormat('de-DE')** schreibt 123,4 statt 123.4. Das Objekt wird einmal außerhalb der Funktion erzeugt, nicht bei jedem Render neu.
- **Reine Funktion ohne React** in `components/measurements.ts`: Kachel und Raumkarte benutzen dieselbe Logik, Formatierung steht nur an einer Stelle.

- **!== null statt Wahrheitswert:** `if (device.powerWatt)` wäre bei 0 W falsch, weil 0 in JavaScript als false gilt. Die Steckdose liefert aber gerade 0 W.

> Falle in JavaScript: 0, '' und NaN sind „falsy“. Wer auf vorhandene Werte prüft, schreibt !== null (oder != null), sonst verschwindet die Anzeige genau bei 0.

9 Tests und ein Fehler, der lehrreich war

Die neuen Tests benutzen genau die echten Rohwerte, z. B. Barlicht mit `cur_voltage = 2271` → 227,1 V und nur einem Ereignis (`turned_off`). Unplausible Werte laufen als `[Theory]` mit mehreren `[InlineData]`: `"83"` als Text, 150, -1, 12.5, true.

Warum drei alte Tests plötzlich rot waren

Die Testfabrik startet die App im Development-Modus. Dort liest ASP.NET Core die **User Secrets des Entwicklers** mit. Am Morgen wurde im Dienst der Leseschutz eingeschaltet (`Auth:RequireLoginForReading = true`), und schon bekamen Tests, die „Lesen ohne Anmeldung“ erwarten, eine 401.

```
private static readonly Dictionary<string, string?> FixedSettings = new()
{
    ["Auth:RequireLoginForReading"] = "false",
    ["Agent:ApiKey"] = "",
};
builder.ConfigureAppConfiguration((_, config) => config
    .AddInMemoryCollection(FixedSettings)     // feste Werte für alle Tests
    .AddInMemoryCollection(Settings));        // Unterklasse darf überschreiben
```

> Tests müssen reproduzierbar sein: gleiches Ergebnis auf jedem Rechner. Alles von außen (Uhrzeit, Konfiguration, Netzwerk) legt der Test selbst fest. Bei der Konfiguration gewinnt die zuletzt hinzugefügte Quelle.

10 Grenzen (ehrlich)

- **Watt-Faktor** ist noch nicht mit einer eingeschalteten Last bestätigt (Barlicht war offline).
- **Kein Verlauf** der Messwerte, nur der letzte Stand.
- **Kein automatischer Abgleich:** Neue Werte gibt es nur nach „Mit Tuya abgleichen“.
- **An/Aus und Tür auf/zu** werden auch bei Offline-Geräten übernommen (dann der letzte Stand). Nur die Messwerte sind gegen alte Werte geschützt.
- **Temperatur** fehlt: Die Sensoren hängen nicht im Tuya-Cloud-Projekt.

11 Kurzantworten für Rückfragen

Frage	Antwort
Warum schickt Tuya 2271 statt 227,1?	Ganze Zahlen plus Faktor: kleiner, ohne Rundungsfehler. Der Empfänger rechnet um.
Woher kennt ihr den Faktor?	Aus den echten Rohwerten: 2271 kann nur 227,1 V Netzspannung sein.
Warum kein Ereignis pro Messwert?	Ereignisse sind Wechsel. Watt schwankt dauernd, das wäre Rauschen.

Frage	Antwort
Was passiert bei Offline-Geräten?	Tuya liefert den alten Stand. Wir übernehmen ihn nicht und zeigen „letzter Stand“.
Warum sind die Spalten nullable?	„Kein Wert“ ist etwas anderes als 0 W. null heißt: Gerät misst das nicht.
Warum !== null im Frontend?	Weil 0 falsy ist. Sonst verschwände die Anzeige bei 0 W.

12 Merksätze

- Rohwert + Faktor = echter Wert. Faktor an echten Daten prüfen, nie raten.
- Ereignis = Wechsel mit Zeitpunkt. Messwert = letzter Stand.
- Offline heißt: Die Cloud kennt nur die Vergangenheit.
- null ist nicht 0: nullable Spalten und !== null im Frontend.
- Tests legen ihre Umgebung selbst fest, sonst sind sie nicht reproduzierbar.